El tema 1 es una introducción , para el examen no entra



el 5 y el 16 no están resueltos pq le molan para el examen

hay que saberse algo de teoría de aqui? → no
los arrays multidimensionales entran? → si
vamos a usar las clases? → no
lo de las funciones anónimas entra?
las tildes en las palabras palíndromas darían igual??? (ej 7, t3) → si

Instalar node

Para ver si está instalado hacemos:

- node –version



VARIABLES

let → declarar variables

- Se usa cuando el valor va a cambiar
- Si es texto lo ponemos entre comillas “”
- Si es numero sin comillas (ya sea decimal o entero)
- true/false (valores que se responden con un verdadero o falso)

```
let nombre = "Iker";
console.log(nombre);
```

No usar las variables var 👈❌

Podemos usar const si el valor de esa variable nunca va a cambiar

console.log → muestra el mensaje en la consola del navegador

```
let nombre = "Iker";
let edad = 17;
let ciudad = "Teruel";
console.log( "Hola me llamo " + nombre + ", tengo " + edad + " años " +
```

```
const nombre = "María"
const edad = 25
const ciudad = "Madrid"

// 1. "Hola María"
const saludo = `Hola ${nombre}`

// 2. "María tiene 25 años"
const presentacion = `${nombre} tiene ${edad} años`
```

Así queda más profesional (interpolación), siempre que usemos interpolación hay que usar comillas simples ``

también se pueden hacer cálculos en el propio mensaje

```
const a = 5
const b = 67

const resultado = `La suma es ${a+b}`
```

"Usa siempre const por defecto. Usa let solo cuando necesites modificar el valor"

OBJETOS *

Un objeto es como una "caja" que guarda información

```
const persona = {
    nombre: "Iker",
    edad: 67,
    email: "ilozanor@iesch.org"
}
```

Acceder al objeto

- Forma antigua

```
const persona = {
  nombre: "Ana",
  edad: 25
}

const nombre = persona.nombre
const edad = persona.edad

console.log(nombre) // Ana
console.log(edad) // 25
```

De esta manera cambiamos el nombre del objeto persona

- Forma nueva (desestructurar) es para sacar los datos

```
const persona = {
  nombre: "Ana",
  edad: 25
}

const { nombre, edad } = persona

console.log(persona) // objeto completo
console.log(nombre)  // Ana
console.log(edad)    // 25
```

Saca las propiedades nombre y edad del objeto persona

También podemos sacar un atributo y cambiarle el nombre

```
// Extrae año y paginas, renombrando paginas como "total_paginas"
const { año, paginas: total_paginas} = libro
```

this

FUNCIONES DE FLECHA

Se define usando la sintaxis de flecha =>

Diferencia entre una función norma y una flecha:

```
//funcion tradicional
function saludar1(nombre) {
  return "Hola " + nombre;
}

console.log(saludar1("Funcion tradicional"));

//funcion de flecha
const saludar2 = (nombre) => {
  return "Hola " + nombre
}

console.log(saludar2("Funcion de flecha"))
```

Tambien esta la funcion de fecha simplificada

- se puede omitir {} y return

```
//funcion de flecha simplificada
const saludar3 = (nombre) => "Hola " + nombre

console.log(saludar2("Funcion de flecha simplificada"))
```

La usaremos cuando solo devolvamos UN valor

En las funcione vamos a poder tener uno, varios o ningún parámetro

```
const sumar = (a, b) => a + b


console.log(sumar(5, 3))  // 8
```

```
const obtenerFecha = () => new Date()
```

- Date es una función que nos vuelve la fecha actual

Si solo tenemos un parámetro no hace falta poner ()

Entonces esto:

JavaScript

```
const saludar = (nombre) => {
  return `Hola ${nombre}`
}
```

es igual que esto:

JavaScript

```
const saludar = nombre => {
  return `Hola ${nombre}`
}
```

Si la función va ha hacer mas de una cosa lo pondremos dentro de {} (lo mismo que el primer ejemplo)

```
//funcion de flecha con  Múltiples líneas
const calcular = (a, b) => {
  const suma = a + b
  const resultado = suma * 2
  return resultado
}

console.log(calcular(3,2))
```

NO tienen su propio this
NO sirven como constructores

ARRAY

Crear un array

```
// Crear un array
const frutas = ["manzana", "pera", "melocoton"]
```

Acceder a un elemento

```
console.log(numeros[2]) // 30
```

Modificar un elemento

```
const numeros = [10, 20, 30, 40]
numeros[1] = 67
console.log(numeros)
```

Añadir y eliminar

```
numeros.push(50)      // añade al final
numeros.pop()         // elimina el último
numeros.unshift(5)    // añade al principio
numeros.shift()       // elimina el primero
```

Todos estos métodos modifican el array original

Tamaño del array

```
const frutas = ["manzana", "pera", "melocoton"]

console.log(frutas)
console.log(frutas.length + " elementos") //3 elementos
```

Con esto obtenemos el último elemento

```
console.log(frutas[frutas.length-1])
```

Array de objetos (muy importante en VUE)

```
const array = [
    { id: 1, nombre: "Teclado", precio: 30 },
    { id: 2, nombre: "Ratón", precio: 20 },
    { id: 3, nombre: "Monitor", precio: 200 }
]

    //mostrar nombres con for-each
array.forEach((a) => {
    console.log(a.nombre)
})
```

Destructuring

aqui hacemos que color1, y color3 sean variables con los datos del array

```
// 2. Desestructura y omite elementos
const colores = ["rojo", "azul", "verde"]
const [color1, , color3] = colores
```

```
console.log(color1, color3)
```

lo llamamos como variables normales

para saltarse valores

```
const colores = ["rojo", "azul", "verde"]
const [color1, , color3] = colores
```

FOR-EACH

Lo usaremos principalmente para recorrer arrays, SOLO PARA ARRAYS

Forma antigua → ❌

```
for (let i = 0; i < numeros.length; i++) {
  console.log(numeros[i])
}
```

Forma nueva → ✅

```
const alumnos = ["hector", "javi", "maria", "dylan"]

alumnos.forEach(a => {
    console.log(a)
})
```

Lo mismo pero con un índice delante del elemento

```
alumnos.forEach( (a, indice) => {
    console.log( `${indice+1} - ${a}`)
})
```

```
1 - hector
2 - javi
3 - maria
4 - dylan
```

Con un **console.log**, también mostraria los elementos pero, pero no pasaría por cada elemento uno a uno

SPREAD

Este es el operador spread → **…**
Es una forma de “expandir” (abrir) un array o un objeto para poder usar sus valores dentro de otro.

“saca todo lo que hay dentro y ponlo aquí”

Con arrays

```
const numeros1 = [1, 2, 3]
const numeros2 = [...numeros1, 4, 5]

console.log(numeros2)
// [1, 2, 3, 4, 5]
```

Copiar arrays

```
const listaCopiada = [...listaNum]
```

en *listaCopiada* se nos guardará una copia del array original (listaNum)

Con objetos

```
const persona = {
    nombre: "Iker",
    edad: "19"
}

const personaCompleta = {
    ...persona,
    ciudad: "Teruel",
    CodigoPostal: 44002,
    email: "ilozano@iesch.org"
}

personaCompleta.email = "berva@iesch"

console.log(personaCompleta)
```

cambiar datos:

```
// 3. Copia un objeto y modifica una propiedad
const persona = { nombre: "Ana", edad: 30 }
const personaActualizada = {...persona, edad: 19}
```

MAP

Crea un nuevo array a partir del original

```
const numeros = [1, 2, 3]
const dobles = numeros.map((i) => i * 2) //i es el iterador

//el resultado lo mete en un nuevo array, el original no lo toca
console.log(dobles)
```

Cuando devolvemos un objeto con **map**, necesitamos paréntesis extra: ({ … })

Nota: Cuando devolvemos un objeto con map, necesitamos paréntesis extra: ({ ... })

FOR-EACH vs MAP

map() crea un nuevo array transformando cada elemento del original
for-each sólo recorre el array y tú decides qué hacer dentro (sin devolver nada por sí solo)

```
const numeros = [3,4,6]

//forEach - solo itera, no devuleve nada
const r1 = numeros.forEach( n => n*2 )
console.log(r1) //esto no devulve nada

//map - tranforma y devulve un numero array
const r2 = numeros.map ( n => n*2 )
console.log(r2)
```

en el map nos devolverá la lista de números*2, en el for necesitaremos otro for para mostrar el resultado

aquí se ve bastante clara la diferencia

```
const array = [
    { id: 1, nombre: "Teclado", precio: 30 },
    { id: 2, nombre: "Ratón", precio: 20 },
    { id: 3, nombre: "Monitor", precio: 200 }
]

    //mostrar nombres con for-each
array.forEach((a) => {
    console.log(a.nombre)
})

    //mostrar nombres con map
const nombres = array.map((a) => a.nombre)
console.log(nombres)
```

```
Teclado
Ratón
Monitor
(3) ['Teclado', 'Ratón', 'Monitor']
```

FILTER

Devuelve todos los elementos que cumplan la condición

- ES PARA ARRAYS
- necesita una función

```
const numeros = [5,12,8,20,3,15]

//necesito numeros mayores que 10
const mayoresDe10 = numeros.filter( n => n>10 )
console.log(mayoresDe10)

//necesito numeros pares
const numerosPares = numeros.filter( n => n%2 === 0)
console.log(numerosPares)
```

```
//alumnos aprobados
const alumnos =[
    {nombre: "Iker", edad:19, nota:8},
    {nombre: "Alonso", edad:19, nota:4},
    {nombre: "Pascual", edad:50, nota:6},
    {nombre: "Andres", edad:1, nota:1}
]

const aprobados = alumnos.filter(a => a.nota >=5)
console.log("ALUMNOS APROBADOS:")

aprobados.forEach((i) =>{
    console.log(`${i.nombre}, ${i.edad} años`)
})
```

FIND

Devuelve el primer elemento que cumple la condición.

- ES PARA ARRAYS
- necesita una función

```
const usuarios = [
  { nombre: "Ana", edad: 17 },
  { nombre: "Carlos", edad: 22 },
  { nombre: "Lucía", edad: 19 },
  { nombre: "Pedro", edad: 22 }
]

const anios = usuarios.find((a) => a.edad===22)
console.log(anios)
```

Aquí no hay que hacer otro for para recorrer los resultados pq solo no se va a dar uno, además NO SE PUEDE USAR FOR-EACH con find

Diferencia filter y find

```
    //ejemplo con FILTER , filter devulve un array
const r1 = usuarios.filter( n=> n.id === 2)
r1.forEach( n=> {
    console.log(n.nombre)
})

    //ejemplo con FIND, find devulve un valor
const r2 = usuarios.find( n=> n.id === 2 )
console.log(r2.nombre)
```

ASINCRONÍAS

La asincronía es la capacidad de ejecutar tareas que tardan tiempo (como peticiones o temporizadores) sin detener el resto del programa, permitiendo que el código siga funcionando mientras esas tareas se completan en segundo plano.

```
//SIN ASINCRONIA: los mensajes se ejecutaran inmediatos
console.log("Inicio")
console.log("Contenido")
console.log("Fin")



//CON ASINCRONIA: Se ejecute incio, luego fin, y 2 seg despues el otro
console.log("Inicio")

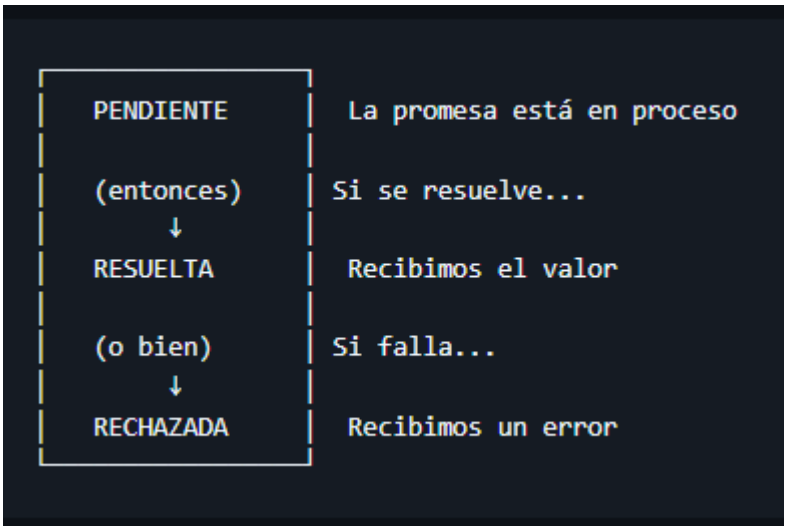
setTimeout(() => {
  console.log("Esto tarda 2 segundos")
}, 2000)

console.log("Fin")
```

PROMESAS *

Una promesa es un objeto que representa un valor que aún no existe, pero existirá en el futuro.

“Te prometo que mañana te traigo un libro. Ahora no lo tengo, pero en el futuro lo tendré”



ejemplo:



```
//crear la promesa
const descragarDatos = new Promise((resolve, reject) => {

    setTimeout(() => {
        resolve({nombre: "Ana", edad: 28 }) //esto es lo que devuelve
    }, 2000) //timepo que va a pasar(2s)
})



//Usar la promesa
descragarDatos
    .then((datos) => { //datos va a ir devolver lo que haya en  al resolve/
        console.log("Datos recibidos: ", datos)
    })
    .catch((error) => {
        console.log("Error: ", error)
    })
```

resolve y reject son funciones:

- resolve(valor) → La promesa se resuelve con un valor
- reject(error) → La promesa se rechaza con un error

.then() → Se ejecuta si la promesa se resuelve
.catch() → Se ejecuta si la promesa se rechaza

ASYNC / AWAIT

async/await no reemplaza las promesas, es una forma más moderna y legible de trabajar con ellas

```
//crear promesa
const esperar25seg = new Promise((resolve) => {
    setTimeout(() => {
        resolve("Los 2 seg han pasado")
    }, 2000)
})



//funcion async
const ejecutar = async() => {
    console.log("Inicio")

    //await pausa aqui 2 seg
    const mensaje = await esperar25seg
    console.log(mensaje)

    console.log("Fin")
}


console.log(ejecutar())
```

FETCH

fetch es la función para descargar datos desde un servidor

```javascript
async function cargarUsuarios() {
  try {
    // Descargar
    const response = await fetch("https://jsonplaceholder.typicode.com/users") //cojo los datos de

    // Convertir a JSON
    const usuarios = await response.json()

    // Mostrar primer usuario
        //console.log(usuarios[0])

    //mostar todos
    usuarios.forEach(a => {
        console.log(`${a.id} - ${a.name}`) //obtengo el id + nombre
    });

  } catch (error) {
    console.log("Error al descargar:", error)
  }
}

cargarUsuarios()
```

```
1 - Leanne Graham
2 - Ervin Howell
3 - Clementine Bauch
4 - Patricia Lebsack
5 - Chelsey Dietrich
6 - Mrs. Dennis Schulist
7 - Kurtis Weissnat
8 - Nicholas Runolfsdottir V
9 - Glenna Reichert
10 - Clementina DuBuque
```

SIGNOS DE COMPARACIÓN

= → Asignar

- const nombre = "Juan" // guarda "Juan" en nombre

== → Comparar valor (no estricto)

- 5 == "5" // true → compara solo el valor, ignora el tipo
- 0 == false // true

=== → Comparar valor Y tipo (estricto)

- 5 === "5" // false → 5 es número, "5" es texto
- 5 === 5 // true
- 0 === false // false

La diferencia entre == y === esq == le da igual qué tipo de dato compares

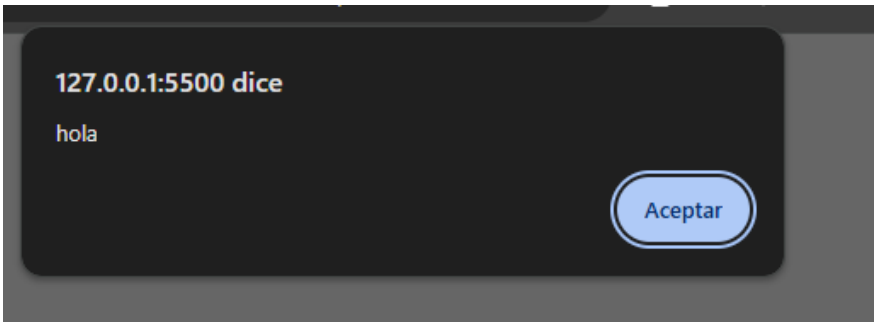
TEMA 2

2. INTRODUCCIÓN

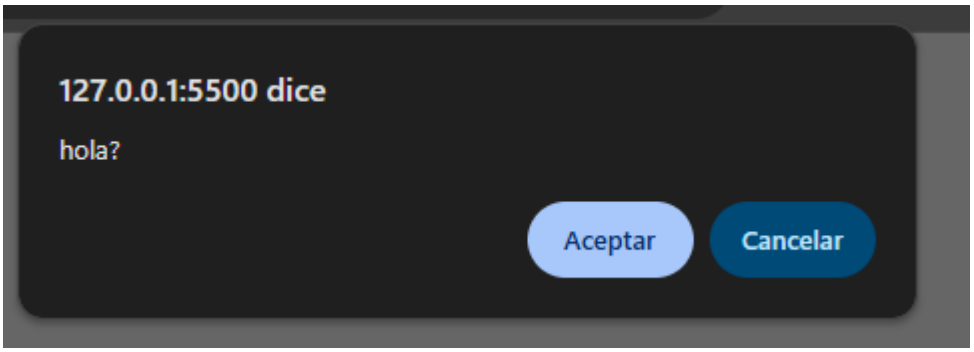
Los comentarios

- //
- /* */

Con **console.log(texto)** se muestra un mensaje por consola
Con **alert(texto)** se muestra un mensaje flotante



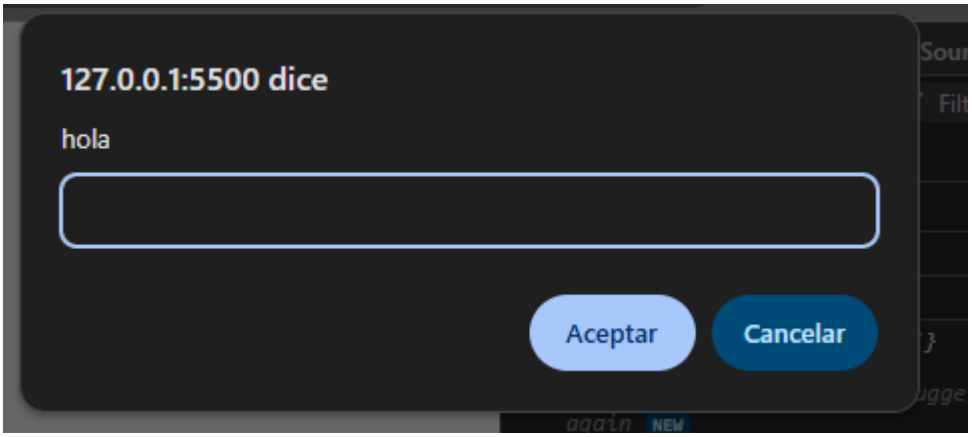
Con **confirm()** se activa un cuadro de diálogo que contiene los botones Aceptar y Cancelar




```
let respuesta
respuesta = confirm("hola?")
alert("tu respuesta a sido " + respuesta) //true o false, segun lo que pul
```

Con **prompt** podemos leer de teclado con una ventana emergente

- siempre devuelve una cadena de texto



document.write(→ : Escribe contenido directamente en el HTML, es como si escribimos algo en al body

3. VARIABLES, CONSTANTES Y ARRAYS

3.1 Variables y Constantes

Usar camelCase → miNombre

let → puedes cambiar el valor
const → no puedes reasignar el valor después de declararlo.
var → accesible desde todos los sitios, forma antigua

let y const solo son accesibles en el bloque que se han declarado

variables sin declarar → como si usaramos var

Cuando haces una concatenación con + entre un string y un number, JavaScript convierte el número a texto automáticamente

```
let numero = 5;

console.log("Tengo " + numero); //aqui el 5 lo interpreta como texto (stri


console.log(numero) //aqui lo intepreta como numero (number)
```

```
let a = "2", b = 5;
console.log( typeof a + " " + typeof b); // string,  number
console.log( a + b ); // nos muestra 25
//si a en vez de ponerlo como string, le quitamos las comillas nos mostrara
```

== → comparación con conversión
=== → comparación sin conversión (tiene q ser mismo tipo y mismo num)

```
console.log( 5 == "5" ) //true
console.log(  5 === "5") //false

console.log("15" < 100) //true, si uno es number y el otro string, convier
console.log( "10" < "9") //true, si los dos son string los compara como te
```

Si los dos valores son string, < y > los comparan como cadenas de texto, no como números, y los compara posición por posición

3.2 ARRAYS

```
// Array definido 1 a 1
let miVector=[]; // let miVector=new Array(); es equivalente
miVector[0]=22;
miVector[1]=12;
miVector[2]=33;
console.log(miVector[1]);
console.log(miVector + " --> "+miVector.length ); // array y tamanyo


//Array definido en una línea
let otroArray=[1,2,"Pascual"]; // Valores dentro del array
console.log(otroArray[2]);
```

con **.push()** metemos nums en el array

Para copiar arrays se usa **...**

```
//copir los arrays
let miArray = { ...otroArray };
console.log(miArray)
```

5. OPERADORES

De asignación

Operador	Descripción
=	Asigna a la variable de la parte izquierda el valor de la parte derecha
+=	Suma los operandos izquierdo y derecho y asigna el resultado izquierdo.
-=	Resta el operando derecho del operando izquierdo y asigna el operando izquierdo.
*=	Multiplica ambos operandos y asigna el resultado al operando izqui
/=	Divide ambos operandos y asigna el resultado al operando izquierd

Aritméticos

Operador	Descripción
+	Suma.
-	Resta.
*	Multiplicación.
/	División.
%	Calcula el resto de una división entera.

Operador	Descripción (Suponiendo x=5)
y = ++x	Primero el incremento y después la asignación. x=6, y=6.
y = x++	Primero la asignación y después el incremento. x=6, y=5.
y = --x	Primero el decremento y después la asignación. x=4, y=4.
y = x--	Primero la asignación y después el decremento x=4, y=5.
y =-x	Se asigna a la variable "y" el valor negativo de "x", pero el valor de no varía. x=5, y= -5.

Comparación (true o false)

Operador	Descripción
===	Compara dos elementos, incluyendo su tipo interno. Si son de dis realiza conversión y devuelve false ya que siempre los considera di **recomendado.**
!==	Compara dos elementos, incluyendo su tipo interno. Si son de dis realiza conversión y devuelve true ya que siempre los considera di **recomendado.**
==	Devuelve el valor *true* cuando los dos operandos son iguales. Si los e de distintos tipos, realiza una conversión. **No está recomendado su**

!=	Devuelve el valor *true* cuando los dos operandos son distintos. Si son de distintos tipos, realiza una conversión. **No está recomendad**
>	Devuelve el valor *true* cuando el operando de la izquierda es mayo derecha.
<	Devuelve el valor *true* cuando el operando de la derecha es meno izquierda.
>=	Devuelve el valor *true* cuando el operando de la izquierda es mayor de la derecha.
<=	Devuelve el valor *true* cuando el operando de la derecha es que el de la izquierda.

Lógicos (true o false)

Operador	Descripción
&&	Y “lógica”. El valor de devolución es true cuando ambos o verdaderos.
||	O “lógica”. El valor de devolución es true cuando alguno de los verdadero o lo son los dos.
!	No “lógico”. Si el valor es true, devuelve false y si el valor es false,

6. ESTRUCTURAS DE CONTROL

6.1 if/else

```
if (condición) {
// bloque de instrucciones que se ejecutan si la condición se
}
else{
// bloque de instrucciones que se ejecutan si la condición no
}
```

```
if (condición1) {
  // bloque que se ejecuta si condición1 se cumple
  if (condición2) {
    // bloque que se ejecuta si condición1 y condición2 se cumplen
  }
  else {
    // bloque que se ejecuta si condición1 se cumple pero condición2 no
  }
}
else {
  // bloque que se ejecuta si condición1 no se cumple
}
```

6.2 Switch

```
switch (operacion){
    case "+":
        document.write((dato1+dato2));
        break;
    case "-":
        document.write((dato1-dato2));
        break;
    case "*":
        document.write((dato1*dato2))
        break;
    case "/":
        document.write((dato1*dato2))
        break;
    default:
        document.write(("Esa opcion no se puede"))

}
```

7.BUCLES

7.1 for

```
for (Inicialización del índice; Condición de prueba; Modificac
índice){
 // ...instrucciones...
}
```

```
//numeros pares
for(i=2;i<=30;i+=2){
    console.log(i)
}
```

7.2 While

Si la condición siempre se cumple, bucle infinito
Si la condición unca se cumple no entrar al while

Solo entra si la condición se cumple

```
while (condición){
//...instrucciones...
}
```

```
let i = 2;
while(i<=30){
    console.log(i)
    i = i+2
}
```

7.2 Do While

Entrar una vez si o si

```
do {
// ...instrucciones...
} while(condición);
```

```
let numS = 10
let num
do {

    num = prompt("Dime un numero")

}while(num!=10)

console.log("Has acertado")
```

7.3 break y continue

En los bucles for, while y do-while se pueden utilizar las instrucciones **break** y **continue**

break → termina el bucle inmediatamente.

- sale del bucle y continúa ejecutando el código que haya después
- sale completamente del bucle, aunque esté dentro de un if

```
//cuando llege al 5 se sale del for
for (let i = 0; i < 10; i++) {
    if (i === 5) {
        break;
    }

console.log(i);
```

```
0
1
2
3
4
```

continue → No termina el bucle, lo que hace es saltarse esa vuelta concreta y continuar con la siguiente iteración.

```
//cuando llegue al 3 da un salto y sigue con el 4
for (let i = 0; i < 6; i++) {
    if (i === 3) {
        continue;
    }

console.log(i);
```

```
0
1
2
4
5
```

8. FUNCIONES

```
function nombrefuncion (parámetro1, parámetro2=valorPorDefecto
// instrucciones
//si la función devuelve algún valor añadimos:
return valor;
```

```
<script>

        function calcularPrecioFinal (precio, descuento){

            desct = precio*descuento/100

            return precio-desct

        }

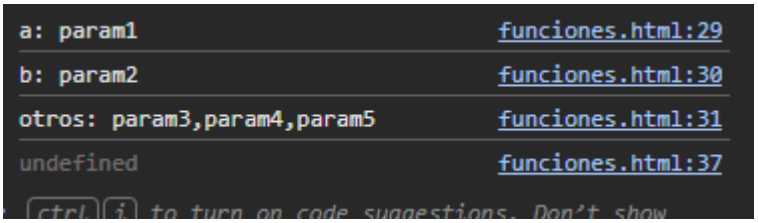

        console.log(calcularPrecioFinal(100, 20))


</script>
```

Usando **...nombreParametro** (el nombre puede ser el que quieras) le decimos que una función pueda recibir tantos parámetros adicionales como quieras.
...masParametros recoge todos los parámetros que sobren y los mete en un array.
El **REST** debe ser el último parámetro

```
function pruebaParREST(a, b, ...masParametros) {
    console.log("a: " + a);
    console.log("b: " + b);
    console.log("otros: " + masParametros);
}


console.log(pruebaParREST("param1", "param2", "param3", "param4", "param5"
```



9. FUNCIONES DE FLECHA

FUNCIONES DE FLECHA

Se define usando la sintaxis de flecha =>

Diferencia entre una función norma y una flecha:

```
//funcion tradicional
function saludar1(nombre) {
  return "Hola " + nombre;
}

console.log(saludar1("Funcion tradicional"));


//funcion de flecha
const saludar2 = (nombre) => {
  return "Hola " + nombre
}

console.log(saludar2("Funcion de flecha"))
```

Tambien esta la funcion de fecha simplificada

- se puede omitir {} y return

```
//funcion de flecha simplificada
const saludar3 = (nombre) => "Hola " + nombre

console.log(saludar2("Funcion de flecha simplificada"))
```

La usaremos cuando solo devolvamos UN valor

En las funciones vamos a poder tener uno, varios o ningún parámetro

```
const sumar = (a, b) => a + b


console.log(sumar(5, 3))  // 8
```

Si solo tenemos un parámetro no hace falta poner ()



Si la función va ha hacer mas de una cosa lo pondremos dentro de {} (lo mismo que el primer ejemplo)

```
//funcion de flecha con  Múltiples líneas
const calcular = (a, b) => {
  const suma = a + b
  const resultado = suma * 2
  return resultado
}

console.log(calcular(3,2))
```

10. EJEMPLOS

Crea un script que genera una pirámide con los números que tenga la siguiente pinta:

1
22

333
4444
55555

```
let num = prompt("Hasta que numero quieres llegar")

for(let i = 1; i<=num; i++){ //controla que numero dibujamos

    for(let j = 1; j <= i; j++){
        document.write(i)
    }
    document.write("<br/>")
}
```

Ej 8 T2

Ejemplo con isNaN

```
let entrada

do{

    entrada = prompt("Dime un numero entero")
    num = Number(entrada) //lo paso a num pq el prompt convierte todo a te

}while(isNaN(num)) //esto para mientras se cunpla la condicion

/*

    isNaN() sirve para comprobar si un valor NO es un número válido.
    isNaN(25)       // false
    isNaN(12.5)     // false
    isNaN("hola")   // true
    isNaN(NaN)      // true

*/
```

Ej 11 T2

11. MÁS COSAS

Sacar el ultimo núm de un num (de 127 sacar el 7)

```
/*
    let numS = "12345";

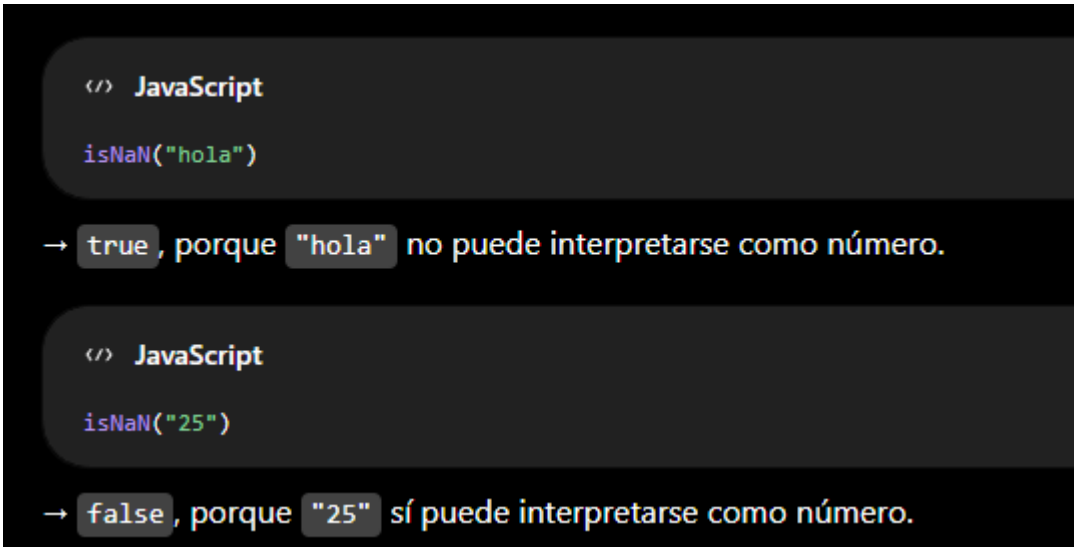
    numS.length → 5 (hay 5 caracteres).
    numS.length - 1 → 4 (la última posición).
    numS[4] → "5" (el carácter que está en esa posición).
*/
```

TEMA 3

1. FUNCIONES PREDEFINIDAS

Son funciones que vienen hechas ya

- **parseInt(cadena)** → convierte una cadena de texto a entero
- **parseFloat(cadena)** → convierte una cadena de texto a decimal
- **isNaN(cadena)** → comprueba si una cadena de caracteres puede ser considerada un número (de false) o no (da true)



- **eval** → interpreta una cadena de texto como código JavaScript y ejecuta ese código

```
let x = 3;
let y = 2;

console.log("x+y")
console.log(eval("x+y"))
```

```
x+y
5
```

2. MÉTODOS STRING

toLowerCase() → Convierte el texto a minúsculas.
toUpperCase() → Convierte el texto a mayúsculas.
concat(cadena) → Une otra cadena al final
charAt(posición) → Devuelve el carácter de una posición.

indexOf(texto) → Busca la primera posición donde aparece un texto
lastIndexOf(texto) → Busca la última posición donde aparece un texto.
replace(texto1, texto2) → Sustituye un texto por otro.
split(separador) → Divide una cadena en partes y devuelve un array.
substring(inicio, fin) → Obtiene una parte de la cadena.

- cuando no se pone el segundo número es de ese número hasta el final

```
// 1. toLowerCase()
// Convierte la cadena a minúsculas
let cadena = "Hola MUNDO";
console.log(cadena.toLowerCase()); // "hola mundo"


// 3. concat(cadena)
// Une una cadena al final de otra
let nombre = "Hola";
console.log(nombre.concat(" Pascual")); // "Hola Pascual"


// 4. charAt(posicion)
// Devuelve el carácter que hay en una posición
let texto = "Pascual";
console.log(texto.charAt(0)); // "P"
console.log(texto.charAt(3)); // "c"


// 5. indexOf(texto)
// Devuelve la primera posición donde aparece el texto
let frase = "Hola Pascual";
console.log(frase.indexOf("Pascual")); // 5
console.log(frase.indexOf("a"));       // 3


// 6. lastIndexOf(texto)
// Devuelve la última posición donde aparece el texto
console.log(frase.lastIndexOf("a")); // 9
```

```js
// 7. replace(texto1, texto2)
// Sustituye un texto por otro
let mensaje = "Hola Juan";
console.log(mensaje.replace("Juan", "Pascual")); // "Hola Pascual"


// 8. split(separador)
// Divide la cadena y crea un array
let datos = "Pascual:Perez:123456";
console.log(datos.split(":")); // ["Pascual", "Perez", "123456"]


// 9. substring(inicio, fin)
// Devuelve una parte de la cadena
let palabra = "Pascual";
console.log(palabra.substring(0, 4)); // "Pasc"
```

3. MÉTODOS DATE

Los meses en Date se numeran del 0 al 11 (siendo el 0 Enero y el 11 Diciembre) mientras que los días si se numeran del 1 al 31. Los días de la semana van de 0 al 6

set/get → son métodos que permiten cambiar el valor de alguna parte de la fecha (set) u obtener el valor de alguna parte de la fecha (get).

- **getMonth()** → Devuelve el mes de la fecha
- **setMonth(mes**) → Cambia el mes de la fecha.
- **getFullYear** → **Dice el año (no usar getYear)**
- ….

toDateString() → Convierte la fecha en una cadena de texto
toGMTString() → Convierte la fecha en una cadena con formato GMT.
toUTCString() → Convierte la fecha en una cadena con formato UTC.

```javascript
// 1. Crear una fecha con la fecha y hora actual
let fechaActual = new Date();
console.log(fechaActual); // Fecha y hora actuales


// 2. Crear una fecha mediante una cadena
let fecha1 = new Date("October 13, 2014 11:13:00");
console.log(fecha1); // Mon Oct 13 2014 11:13:00


// 3. Crear una fecha indicando año, mes, día, hora, minutos, segundos y
let fecha2 = new Date(99, 5, 24, 11, 33, 30, 0);
console.log(fecha2); // Thu Jun 24 1999 11:33:30


// 4. Crear una fecha indicando año, mes y día
let fecha3 = new Date(99, 5, 24);
console.log(fecha3); // Thu Jun 24 1999 00:00:00
```

```
// GET / SET DEL MES
let fecha = new Date(2026, 8, 28, 10, 30, 0); //año, mes, día, hora, minut
console.log(fecha.getMonth()); // 8 → septiembre
fecha.setMonth(11);
console.log(fecha.getMonth()); // 11 → diciembre


// GET / SET DEL DÍA DEL MES
console.log(fecha.getDate()); // 28
fecha.setDate(15);
console.log(fecha.getDate()); // 15


// FECHA COMO TEXTO
console.log(fecha.toDateString()); // Tue Dec 15 2026


// FECHA EN GMT
console.log(fecha.toGMTString()); // Fecha en formato GMT


// FECHA EN UTC
console.log(fecha.toUTCString()); // Fecha en formato UTC
```

getDay() → Devuelve el día de la semana, con números del 0 al 6

- 0 domingo
- 1 lunes
- …..

getDate() → Devuelve el día del mes, con números del 1 al 31.

tanto los mese como los días de la semana empiezan por 0 no

Ejemplos

```
let fechaDeHoy = new Date() //fecha actual
let anio = fechaDeHoy.getFullYear() //saco el año de la fecha
let mes = fechaDeHoy.getMonth() //saco el mes de la fecha
let fechaCompletaDia1 = new Date(anio, mes, 1) //fecha completa del dia 1
let diaSemana = fechaCompletaDia1.getDay() //dia de semana de esa fecha, m
let fechaCompletaDiaU = new Date(anio, mes + 1, 0) //fecha completa del ul
let ultimoDia = fechaCompletaDiaU.getDate()//saco el ultimo dia del mes, m
```

TRABAJAR CON TIEMPO (HORAS Y MINS)

Crear una h nueva → con new Date creamos una fecha y hora, la fecha nos dará igual, usamos solo la h

- new Date(año, mes, día, hora, minutos)

Obtener las horas y los minutos

- getHours() → Obtiene las horas.
- getMinutes() → Obtiene los minutos.

Sumar horas o minutos (ejemplo)

```
let horaInicio = new Date(2007, 3, 1, 15, 45)
```
//creo la hora

```
horaInicio.setMinutes(horaInicio.getMinutes() + 55)
```

- obtener los mins actuales y le sumó 55 y con el set lo actualizo
- Si los minutos superan 59, JavaScript actualiza automáticamente las horas

→ *Ejemplo completo en RepasoExamen > Horario*

4. MÉTODOS DE ARRAY

join() → Une todos los elementos del array en una cadena de texto, usando el separador indicado.
push() → Añade uno o varios elementos al final del array.
pop() → Elimina y devuelve el último elemento del array.
reverse() → Invierte el orden de los elementos.

sort() → Ordena los elementos alfabéticamente. Con números puede dar resultados inesperados.
slice() → Obtiene una parte del array sin modificar el original.
set() → Elimina duplicados
test() → comprueba si un texto cumple un regex

```javascript
// ARRAY DE EJEMPLO
let numeros = [1, 2, 3, 4, 5];


// 1. JOIN()
// Une todos los elementos en una cadena
let texto = numeros.join("-");
console.log("JOIN:", texto);

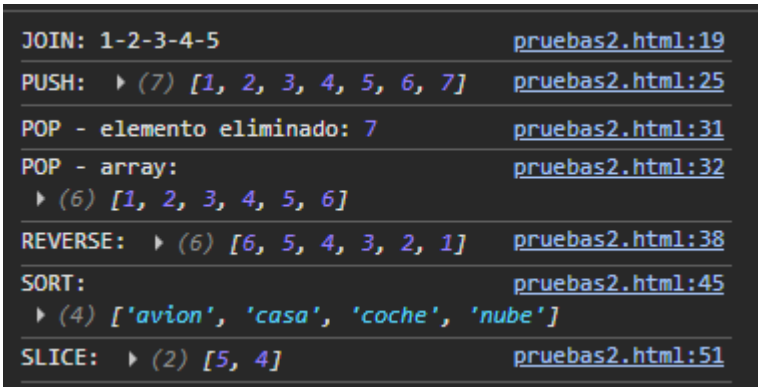

// 2. PUSH()
// Añade elementos al final
numeros.push(6, 7);
console.log("PUSH:", numeros);


// 3. POP()
// Elimina y devuelve el último elemento
let eliminado = numeros.pop();
console.log("POP - elemento eliminado:", eliminado);
console.log("POP - array:", numeros);


// 4. REVERSE()
// Invierte el orden del array
numeros.reverse();
console.log("REVERSE:", numeros);


// 5. SORT()
// Ordena alfabéticamente
let numeros2 = ["casa", "coche", "avion", "nube"];
numeros2.sort();
console.log("SORT:", numeros2);


// 6. SLICE()
// Coge una parte del array
let parte = numeros.slice(1, 3); //coga el elemento de la posicion 1 y 3
console.log("SLICE:", parte);
```



Quitar duplicados y ordenar

```
let array = [4,0,3,4,7,10,3,5,8,1,8,8,0,2,3,1,2,5,7,3,2,5,1]

let arrayNuevo = [...new Set(array)] //elimino los duplicados

arrayNuevo.sort((a, b) => a-b) //ordenar


document.write(array)
document.write("<br/>" + arrayNuevo)
```

Set no permite elementos repetidos.
Pero usamos … para convertir la respuesta del set otra vez a array

- Set(4) { 1, 2, 3, 4 }

Regex

```
let email = "info@elcorreo.com"

let patron = /^[A-Za-z]+\@[A-Za-z]+\.[A-Za-z]{3,}$/


let arroba= email.indexOf("@")
let ultimo = email.length

if(patron.test(email)){
    document.write("El email es correcto" + "<br/>")
    document.write("nombre de usuario --> " + email.substring(0, arroba) +
    document.write("nombre de dominio --> " + email.substring(arroba+1, ul
}else{
    document.write("El email no es corrceto")
}
```

```
let fecha = prompt("Dime una fecha")

let patron = /^\d{2}\/\d{2}\/\d{4}$/

if (patron.test(fecha)){
    document.write("El formato es correcto")
}else{
    document.write("El formato no es correcto")
}
```

test() → comprueba si un texto cumple un regex y devuelve true o false
Solo hay que poner una \ (no \\d)

5. MÉTODOS DE MATH

Constantes de Math
Math.E → almacena el número de Euler.
Math.PI → almacena el número π (Pi).
Math.LN2 → almacena el logaritmo neperiano de 2.
Math.LN10 → almacena el logaritmo neperiano de 10.
Math.LOG2E → almacena el logaritmo en base 2 de E.

Math.LOG10E → almacena el logaritmo en base 10 de E.

Redondeos

Math.floor(número) → redondea hacia abajo al entero más cercano.

- Math.floor(4.8) → 4

Math.ceil(número) → redondea hacia arriba al entero más cercano.

- Math.ceil(4.2) → 5

Math.round(numero) → redondea al entero más cercano.

- Math.round(4.6) → 5

Operaciones matemáticas

Math.abs(número) → devuelve el valor absoluto, es decir, elimina el signo negativo.

- Math.abs(-8) → 8

Math.max(x, y) → devuelve el mayor de los números indicados.

- Math.max(5, 9) → 9

Math.min(x, y) → devuelve el menor de los números indicados.

- Math.min(5, 9) → 5

Math.pow(x, y) → devuelve x elevado a y.

- Math.pow(2, 3) → 8

Math.random() → genera un número aleatorio entre 0 y 1, con decimales.

- Math.random() → 0.734...

Math.sqrt(numero) → devuelve la raíz cuadrada de un número.

- Math.sqrt(25) → 5
-

```
// FLOOR
console.log(Math.floor(4.8)); // 4

// CEIL
console.log(Math.ceil(4.2)); // 5

// ROUND
console.log(Math.round(4.6)); // 5

// ABS
console.log(Math.abs(-10)); // 10

// MAX
console.log(Math.max(5, 9)); // 9

// MIN
console.log(Math.min(5, 9)); // 5

// POW
console.log(Math.pow(2, 3)); // 8

// SQRT
console.log(Math.sqrt(25)); // 5

// RANDOM
console.log(Math.random()); // Ej: 0.73452...


// Número aleatorio entre 1 y 10
let numero = parseInt((Math.random() * 10) + 1);
console.log(numero);
```

7. MÉTODOS DE WINDOW

Controla elementos de lo que ocurre en la “ventana” del navegador.
Los métodos del tema anterior cómo alert, prompt, etc. forman parte del objeto Window

setTimeout(function, tiempo) → ejecuta una función una sola vez después del tiempo indicado en milisegundos.

- **clearTimeout(id)** → cancela un setTimeout utilizando el identificador que devuelve.

setInterval(function, tiempo) → ejecuta una función repetidamente, cada vez que pasa el tiempo indicado.

- **clearInterval(id)** → cancela un setInterval utilizando el identificador que devuelve.

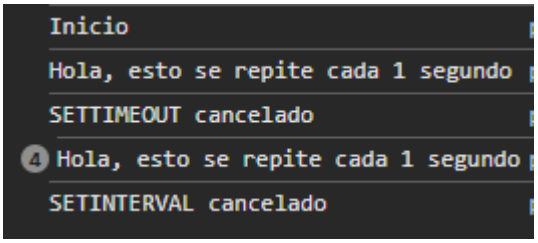
```
console.log("Inicio");


// SETTIMEOUT
let idA = setTimeout(() => {
    console.log("Esto se ejecutará después de 5 segundos");
}, 5000);

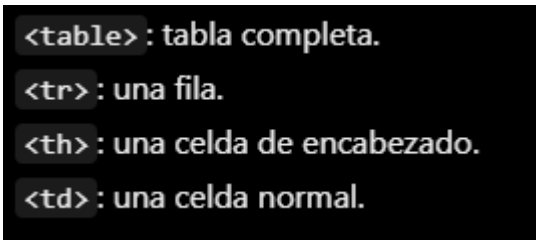

// Paramos el SETTIMEOUT antes de que pasen los 5 segundos
setTimeout(() => {
    clearTimeout(idA);
    console.log("SETTIMEOUT cancelado");
}, 2000);


// SETINTERVAL
let idB = setInterval(() => {
    console.log("Hola, esto se repite cada 1 segundo");
}, 1000);


// Paramos el SETINTERVAL después de 5 segundos
setTimeout(() => {
    clearInterval(idB);
    console.log("SETINTERVAL cancelado");
}, 5000);
```



Para lo del calendario



TEMA 4

1. INTRODUCCIÓN

Representa un documento HTML como un árbol de nodos, permitiendo acceder, recorrer y modificar sus elementos mediante código

3. FUNCIONES PARA LOCALIZAR ELEMENTOS

Si tenemos un elemento html podemos seleccionarlos para hacer cualquier cosa con ellos

1. getElementById()

Buscar un elemento por su id

```
<div id="miDiv">

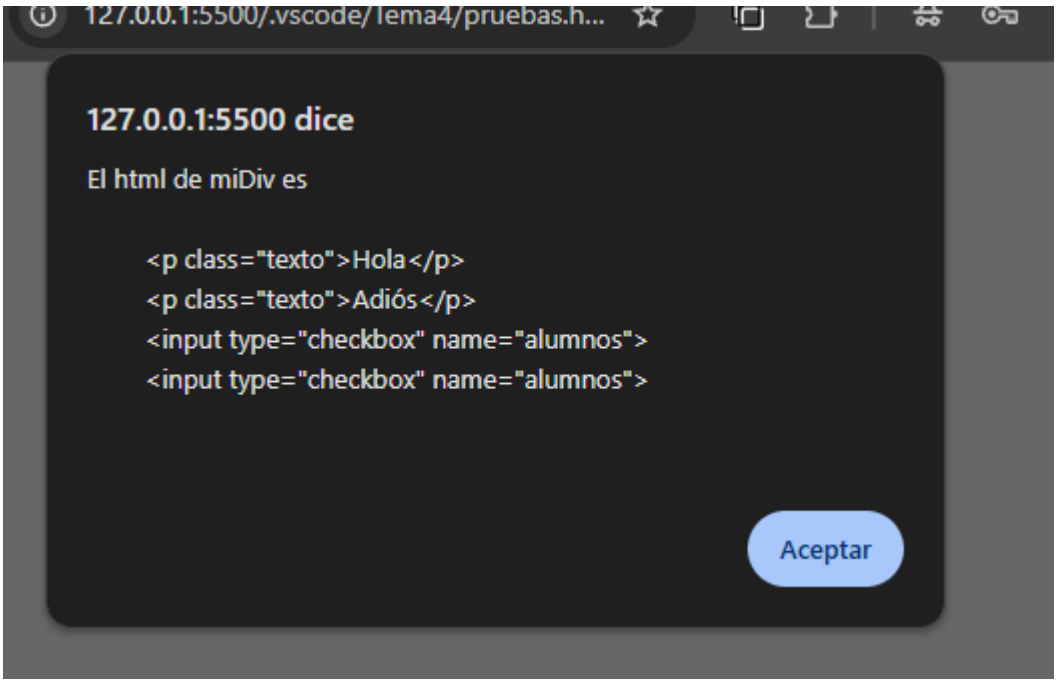
    <p class="texto">Hola</p>
    <p class="texto">Adiós</p>
    <input type="checkbox" name="alumnos">
    <input type="checkbox" name="alumnos">

</div>



<script>

    //busco el id de ese elemeto y guardo el elemento en myDiv
    var myDiv = document.getElementById("miDiv");//document: representa todo el documento HTML
    alert("El html de miDiv es" + myDiv.innerHTML);
    //innerHTML -->  permite obtener o modificar todo el contenido HTML que hay dentro de un elemento.
```



Con ***innerHTML*** no solo podemos mostrar el texto, también lo podemos:

- modificar
    - `myDiv.innerHTML = "<p>Hola Iker</p>”`
- Añadir/insertar
    - `myDiv.innerHTML += "<p>Otro párrafo</p>";`
- Eliminar
    - `myDiv.innerHTML = "";`

```html
<body>

    <p id="texto">hola</p>
    <p id="texto2">Buenas</p>



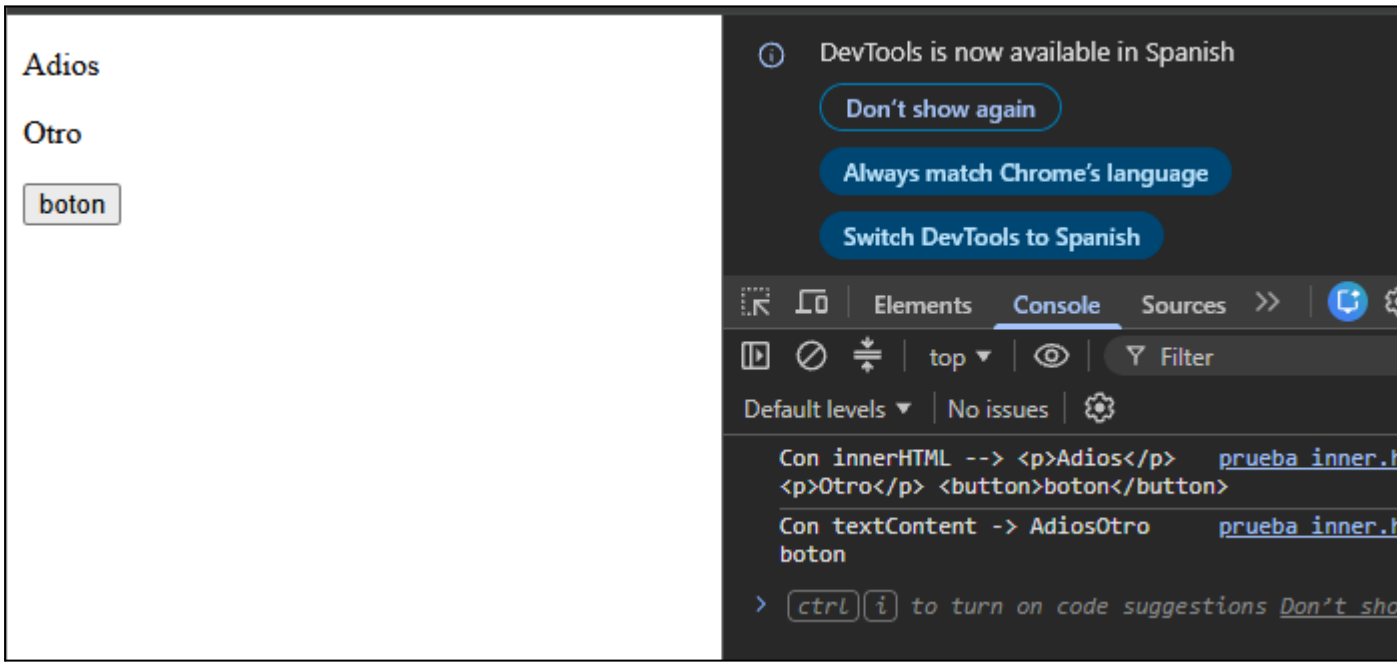
    <script>


        var parrafo = document.getElementById("texto")
        var parrafo2 = document.getElementById("texto2")

        parrafo.innerHTML = "<p>Adios</p>" //modificar
        parrafo.innerHTML += "<p>Otro</p> <button>boton</button>" //añadir
        parrafo2.innerHTML = "" //eliminar
        console.log("Con innerHTML --> " + parrafo.innerHTML) //ver
        console.log("Con textContent -> " + parrafo.textContent) //ver

    </script>

</body>
```



innerHTML → devuelve el contenido incluyendo las etiquetas HTML.
textContent → devuelve solo el texto, sin las etiquetas.

2. getElementsByTagName(etiqueta)

Busca los elementos que estén dentro de x etiqueta
Devuelve un array con todos los elemento

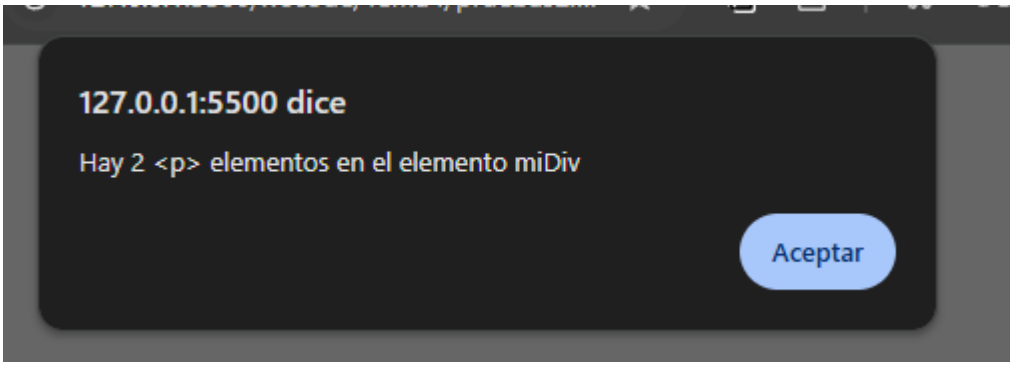
```
var losP = myDiv.getElementsByTagName("p");
```

- Busca todos los elementos <p> que estén dentro de myDiv y los guarda en losP

```
var myDiv = document.getElementById("miDiv");

var losP = myDiv.getElementsByTagName("p");
var num = losP.length; //número de elementos encontrados

alert("Hay " + num + " <p> elementos en el elemento miDiv");
```



Para acceder a uno de los elementos

```
alert("En el primer P el HTML asociado es" +losP[0].innerHTML)
```

3. getElementsByName(nombre)

Buscar elementos por su atributo **name**
Devuelve un array con todos los elemento

```html
<div id="miDiv">

    <p class="texto">Hola</p>
    <p class="texto">Adiós</p>
    <input type="checkbox" name="alumnos">
    <input type="checkbox" name="alumnos">
    <input type="radio" name="alumnos">

</div>



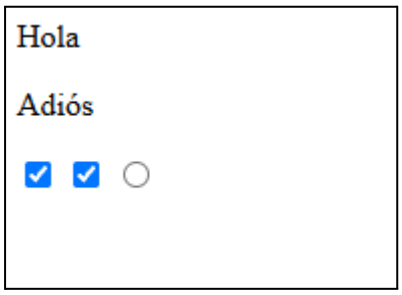

<script>

    //en la x guardo los elementos que name="alumnos
    var x = document.getElementsByName("alumnos");

    //recorro el array de resultados
    for (let i = 0; i < x.length; i++) {

        //si el resultado es un checkbox lo marco
        if (x[i].type == "checkbox"){
            x[i].checked = true
        }

    }
```



getElementsByClassName (clase)

Lo mismo que el anterior pero busca según su **clase**
Devuelve un array con todos los elemento

```html
<div id="miDiv">

    <p class="texto">Hola</p>
    <p class="texto">Adiós</p>
    <p class="importante">Cuidado</p>

</div>



<script>

    //busca los de la clase importante
    var elementos = document.getElementsByClassName("importante");



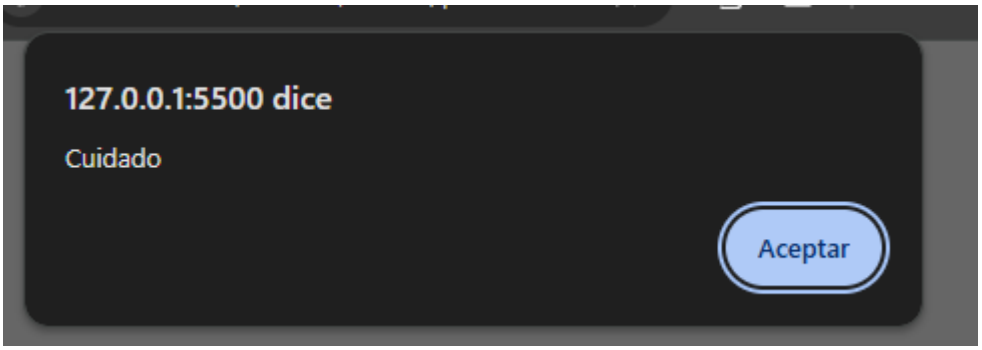
    for (let i = 0; i < elementos.length; i++) {

        alert(elementos[i].innerHTML);

    }



</script>
```



4. FUNCIONES PARA CREAR/ELIMINAR NODOS

Borrar el elemento o crear nuevos

1. removeChild(nodo)

permite eliminar un elemento hijo del documento. Se aplica sobre su elemento padre
Solo se puede eliminar mediante el elemento padre, porque es el padre quien lo borra.

```
<div id="miDiv">
    <p id="miParrafo">Hola, ¿qué tal?</p>
</div>



<script>


    var parrafo = document.getElementById("miParrafo");

    //obtengo el elemento padre cuyo hijo tiene ese id y lo eilimino
    parrafo.parentNode.removeChild(parrafo);
    //parrafo.textContent = ""; //vacio el texto peo no lo borro
```

parentNode → propiedad que devuelve el elemento padre de un elemento

Si en vez de borrarlo lo queremos ocultar usamos **display**
De esta manera ya no ocupan espacio

```
function ocultar(elemento) {

    elemento.style.display = "none" //lo oculta
    console.log(elemento.innerHTML + " --> ocultado")

}



var textoOculto = document.getElementsByClassName("texto")
function reaparacer() {

    for (let i = 0; i < textoOculto.length; i++) {

        //console.log(textoOculto[i].innerHTML)

        if (textoOculto[i].style.display == "none") {

            textoOculto[i].style.display = "block" //lo vuelve a mostrar
        }


    }
```

Si queremos ocultarlos pero que ocupen su espacio usamos ***visibility***

```
function ocultar(elemento) {

    elemento.style.visibility  = "hidden" //lo oculta, pero sigue ocupoanfo
    console.log(elemento.innerHTML + " --> ocultado")

}



var textoOculto = document.getElementsByClassName("texto")
function reaparacer() {

    for (let i = 0; i < textoOculto.length; i++) {

        //console.log(textoOculto[i].innerHTML)

        if (textoOculto[i].style.visibility  == "hidden") {

            textoOculto[i].style.visibility  = "visible" //lo vuelve a most
        }
```

Métodos de cada uno

1. `display`

Controla cómo se muestra un elemento y cómo ocupa espacio en la página.

Valor	Funcionamiento
`none`	Oculta el elemento y no ocupa espacio.
`block`	Se muestra como bloque y ocupa todo el ancho disponible.
`inline`	Se muestra en línea y solo ocupa el espacio necesario.
`inline-block`	Se muestra en línea, pero permite establecer ancho y alto.
`flex`	Activa el modelo de diseño Flexbox.
`grid`	Activa el modelo de diseño Grid.

2. `visibility`

Controla si un elemento es visible o no.

Valor	Funcionamiento
`visible`	El elemento es visible (valor predeterminado).
`hidden`	Oculta el elemento, pero mantiene su espacio.
`collapse`	En filas o columnas de tablas, permite ocultarlas y ajustar el espacio que ocupan.

2. appendChild(nodo)

Lo contrario que la de antes, esta añade un elemento dentro de otro elemento que ya existe.
Lo podemos crear dentro del body y así no hace falta tener un elemento pare

```html
<body>


    <ul id="miLista">
        <li>Leche</li>
        <li>Pan</li>
    </ul>



    <script>

        //creo un elemento
        //pude ser en mayuscilas o minusculas
        var nuevoNodo = document.createElement("LI");

        //creo un texto
        var nodoTexto = document.createTextNode("Agua");

        //al elemeto de antes de asigno el texto (<li>Agua</li>)
        nuevoNodo.appendChild(nodoTexto);

        //busca la lista con ese id y añade el elemento
        document.getElementById("miLista").appendChild(nuevoNodo);

    </script>

</body>
```

Todos se creen igual menos estos dos, que necesita un input e indicarle de qué tipo son

```
<script>


    let check = document.createElement("input")
    check.type = "checkbox"
    let texto1 = document.createTextNode("checkbox")//texto del check


    let radio = document.createElement("input")
    radio.type = "radio"
    let texto2 = document.createTextNode("radio")//texto del radio


    document.body.appendChild(radio);
    document.body.appendChild(texto1);

    document.body.appendChild(document.createElement("br"));//espacio entr


    document.body.appendChild(check);
    document.body.appendChild(texto2);



</script>
```

3. setAttribute()

Para modificar, consultar, eliminar o reemplazar atributos de los elementos
Modifica sus atributos (class, type…), inner modifica el texto

Modificó la clase del DIV

```
<body>

    <div id="miCaja" class="caja"></div>


    <script>


        //obtengo el elemento
        var caja = document.getElementById("miCaja");

        //ahora el div tiene la class "caja naranja"
        caja.setAttribute("class", "caja naranja");

        //tambien lo podemos hacer asi
        caja.className = "caja naranja";


        //obtener el valor del atributo y lo muestro
        var clase = caja.getAttribute("class");
        alert(clase);
```

class indica que atributo queremos modificar

Eliminar el atributo

```
//elimino el atributo class
caja.removeAttribute("class");
```

Reemplazo elementos
No solo reemplaza el texto, sino el elemento entero

```html
<div id="miDiv">
    <p>Hola</p>
</div>


<script>


    //obtengo el elemento
    var div = document.getElementById("miDiv");

    //creo u nnuevo elemento y le pongo el texto "Adiós"
    var nuevo = document.createElement("p");
    nuevo.textContent = "Adiós";

    //remplazo el parrafo antiguo (<p>Hola</p>) por el nuevo (<p>Adiós</p
    div.replaceChild(nuevo, div.firstElementChild);
```

5. EVENTOS

Permiten que una página web reaccione a las acciones del usuario.

Estos son los principales:

Evento	Cuándo se ejecuta
`onfocus`	Cuando un elemento obtiene el foco (por ejemplo, pulsas dentro de un input).
`onblur`	Cuando un elemento pierde el foco.
`onchange`	Cuando cambia el valor de un elemento.
`onclick`	Cuando haces clic.
`ondblclick`	Cuando haces doble clic.
`onkeydown`	Cuando pulsas una tecla.
`onkeyup`	Cuando sueltas una tecla.
`onkeypress`	Al pulsar una tecla (obsoleto actualmente).
`onload`	Cuando se carga una página o recurso.
`onunload`	Cuando se abandona una página (con limitaciones actuales).
`onmousedown`	Cuando pulsas un botón del ratón.
`onmouseup`	Cuando sueltas un botón del ratón.
`onmouseover`	Cuando entras con el ratón en un elemento.
`onmouseout`	Cuando sales con el ratón de un elemento.
`onsubmit`	Cuando envías un formulario.
`onreset`	Cuando reinicias un formulario.
`onselect`	Cuando seleccionas texto.
`onresize`	Cuando cambias el tamaño de la ventana.

Importante: `onkeydown` detecta cuando pulsas una tecla y `onkeyup` cuando la sueltas.

mousemove → al mover el ratón por la pantalla
change → Se ejecuta cuando cambia el estado o el valor del elemento (sirve para checkBox y radio)

si el evento se usa en el html hay que poner un “on” delante

MANEJO DE EVENTOS

Maneras de poner un evento

1. Poner el evento directamente en la etiqueta html

```
<!--cuando pulsemos en el boton  sale el mensaje-->
<input type="button" value="Pulsar" onclick="alert('Hola mundo');">
```

Esto solo vale para cosas muy cortas

2. En la etiqueta HTML llamando a una función

```
 <!--cuando pulsemos en el boton  sale el mensaje-->
<input type="button" value="Pulsar 2" onclick="miFuncion()">



<script>



    function miFuncion() {
        alert("Hola mundo");

    }

</script>
```

3. Con addEventListener

Esta es la opciones más recomendable

```html
<button id="boton">Nuevo numero</button>



<script>

    let lista = [];

    document.getElementById("boton").addEventListener("click", function(){

        //genra un num aleatorio son decimales(para eso es el floor)
        //el *10 es para q sea enre 1 y 10
        let num = Math.floor(Math.random() * 10);

        lista.push(num)

        console.log(lista)

    })

</script>
```

Aquí el evento no hay que ponerle “on” delante

Hay 3 maneras de pasar el segundo parámetro:

```javascript
// a) Anónima (como en tu ejemplo)
boton.addEventListener("click", function() { ... });

// b) Función flecha
boton.addEventListener("click", () => { ... });

// c) Función con nombre, aparte y sin ()
boton.addEventListener("click", saludar);
```

Ejemplo:

Si por ejemplo recogemos los datos con un *getElementsByClassName* (que nos devuelve un array) primero tenemos que recorrer un array y después poner el evento

```
for (let i = 0; i < p.length; i++) {

    p[i].addEventListener("click", function () {
        this.style.display = "none"
    })

}
```

4. Con la propiedad del evento

```
<body>



    <button id="boton">Otro boton</button>

    <script>


        let boton = document.getElementById("boton");


        function saludar() {
            alert("Hola");
        }

        boton.onclick = saludar;




    </script>

</body>
```

No hay que ponerlo () cuando llamamos a la función
Aquí siempre “on” delante del evento

También puede ser así y nos ahorramos 1 línea

```
function mostrarMensaje() {
    alert("Hola");
}

document.getElementById("miObjeto").onclick = mostrarMensaje;
```

USO DEL THIS EN EVENTOS

this hace referencia al elemento HTML que está ejecutando el evento
this significa, en este caso, «este mismo elemento».

```
<div id="cont"
    onmouseover="this.style.borderColor='black';"
    onmouseout="this.style.borderColor='red';">

    Contenidos
</div>
<!--
    onmouseover: cuando pasamos el ratón por encima, el borde se vuelve n
    onmouseout: cuando quitamos el ratón, el borde se vuelve rojo
    this: hace referencia al propio div
-->
```

Sin this seria asi (ponerlo en el script)

```
document.getElementById("cont").style.borderColor = "black";
```

Pero utilizando **this** no necesitamos buscar el elemento por su ID.

Si uso por ejemplo *getElementsByClassName()* aunque solo pulse uno se ocultan todos, de esta forma solo se oculta/borra al que le doy click/doble click

```html
<p class="texto" onclick="ocultar(this)" ondblclick="borrar(this)">Hola</
<p class="texto" onclick="ocultar(this)" ondblclick="borrar(this)">Buenas
<p class="texto" onclick="ocultar(this)" ondblclick="borrar(this)">Noches


<script>


    //this: representa el elemento HTML sobre el que has hecho clic


    function borrar(elemento) {

       elemento.remove();
       console.log("eliminado")

    }



      function ocultar(elemento) {

        elemento.style.display = "none"
        console.log("ocultado")

    }
</script>
```

Ejemplo:

```html
<h3>FORMA 1</h3>
<p onclick="funcion(this)">1</p>
<p onclick="funcion(this)">2</p>
<p onclick="funcion(this)">3</p>

<br>

<h3>FORMA 2</h3>
<p class="texto">1</p>
<p class="texto">2</p>
<p class="texto">3</p>



<script>

    //FORMA 1
    function funcion(elmento) {

        elmento.style.display = "none"

    }



    //FORMA 2
    let p = document.getElementsByClassName("texto")



    for (let i = 0; i < p.length; i++) {

        p[i].addEventListener("click", function () {
            this.style.display = "none"
        })

    }
```

OBTENER INFORMACIÓN DEL OBJETO EVENT

Cuando ocurre un evento, el navegador proporciona automáticamente un objeto llamado **event**, que contiene información sobre lo que ha sucedido

Tipos de eventos

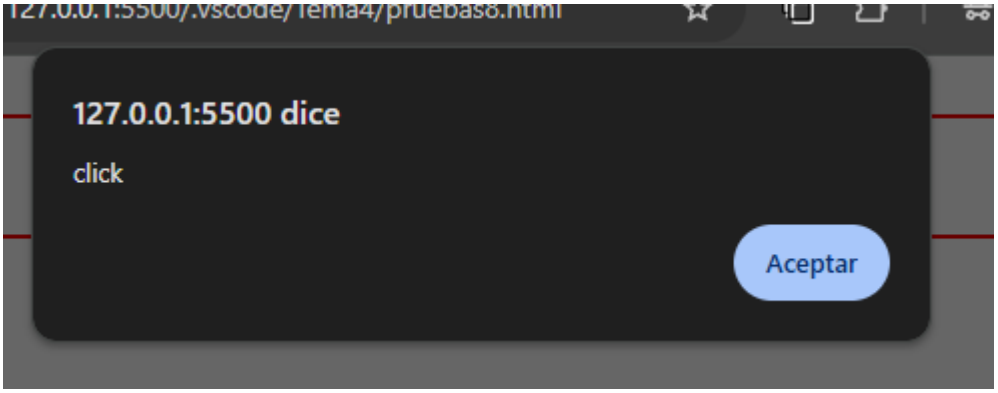
Propiedades principales de event

Propiedad	Qué devuelve
`type`	Tipo de evento (`click`, `keyup`, etc.).
`keyCode`	Código numérico de la tecla (obsoleto actualmente).
`clientX`	Posición horizontal del ratón respecto a la ventana.
`clientY`	Posición vertical del ratón respecto a la ventana.
`screenX`	Posición horizontal del ratón respecto a la pantalla.
`screenY`	Posición vertical del ratón respecto a la pantalla.

Con el **.type** nos indica qué evento ha ocurrido

```
function mostrarMensaje2(evento) {
    alert(evento.type);
}

document.getElementById("miObjeto").onclick = mostrarMensaje2;
```



Si sueltas una tecla, muestra qué tecla has soltado.
Si haces clic, muestra las coordenadas del ratón dentro de la ventana.

```
function mostrarMensaje3(evento) {
    if (evento.type == "keyup") {
        alert(evento.key);
    }
    else if (evento.type == "click") {
        alert(evento.clientX + " " + evento.clientY);
    }
}

document.getElementById("miObjeto").onclick = mostrarMensaje3;
document.onkeyup = mostrarMensaje3;
```

Ejemplo con keypress

```
<body>

  <ul id="lista">

  </ul>


  <script>

    let letra
    let letraM


    document.addEventListener("keypress", function (evento) {


        letra = evento.key
        letraM = letra.toUpperCase()
```

```
        /*
            keypress: es el evento que se activa cuando pulsas una tecla.
            evento.key: obtiene cuál es la tecla que has pulsado.
        */

          var letras = ['T', 'R', 'W', 'A', 'G', 'M', 'Y', 'F', 'P', 'D', 'X', 'B', 'N', 'J', 'Z', 'S', 'Q',
'V', 'H', 'L', 'C', 'K', 'E', 'O'];

        let resto

            let texto2 = document.createTextNode("----------DNI CON " + letraM +
"-------------------------")
        document.getElementById("lista").appendChild(texto2)

        for (let i = 1; i < 9999; i++) {

            resto = i % 23
            let letraDNI = letras[resto]


            if (letraDNI == letraM) {

                //document.write(i + letraDNI + " - ")

                let elemento = document.createElement("LI")
                    let texto = document.createTextNode(i.toString().padStart(4, "0") +
letraDNI) //.padStart(4, "0") --> se asegura que i tenga 4 cifras y si  no te pone un
                elemento.appendChild(texto)
                document.getElementById("lista").appendChild(elemento)
            }


        }


    })
```

```
  </script>

</body>
```

6. MÁS COSAS

INTERVALOS

setInterval() → Ejecuta una función repetidamente cada cierto tiempo.
clearInterval() → Detiene un intervalo.

- dentro de los () pondremos un identificador, que será el del setInterval

```
<button id="boton1">Comenzar Saludos</button>
<button id="boton2">Parar Saludos</button>

<script>

    document.getElementById("boton1").onclick = empezar
    document.getElementById("boton2").onclick = acabar



    function saludo() {
        alert("Hola")
    }


    let intervalo
    function empezar() {
        intervalo = setInterval(saludo, 5000)
    }



    function acabar() {
        clearInterval(intervalo)
    }
```

PADSTART()

Se asegura que un num tengo las cifras que le dices y si no es así te añade el num que le digas, hay que convertir el número primor a string, ya que este método trabaja con string

```
if (letraDNI == letraM) {

    //document.write(i + letraDNI + " - ")

    let elemento = document.createElement("LI")
    let texto = document.createTextNode(i.toString().padStart(4, "0") + let
    document.getElementById("lista").appendChild(elemento)
}
```

TOGGLE

sirve para añadir una clase CSS si no la tiene o quitarla si ya la tiene

```html
<head>
    <meta charset="UTF-8">
    <title>Palabra reservada this</title>
    <style>
        .caja {
            width: 200px;
            height: 200px;
            cursor: pointer;
            background: #226fc1;
            margin: 10px;
            display: inline-block;
            text-align: center;
            font-size: 30px;
            color: #fff;
            font-family: Arial;
        }

        .negro {
            background: #000;
        }
    </style>
</head>

<body>
    <div class="caja">1</div>
    <div class="caja">2</div>
    <div class="caja">3</div>
    <div class="caja">4</div>
    <div class="caja">5</div>


    <script>
        function cambiar() {
            this.classList.toggle('negro');
            //this.classList.add('negro');
        }

        var cajas = document.getElementsByClassName('caja');

        for (var i = 0; i < cajas.length; i++) {
            cajas[i].addEventListener('click', cambiar);
        };

    </script>
```

Cuando haces clic en una caja:

1. Si NO tiene la clase negro, se la añade.
2. Si YA tiene la clase negro, se la quita.

classList permite consultar, añadir, eliminar y modificar las clases CSS de un elemento HTML.

ELEMENTOS HIJOS

con children saco todos los elementos de un elemento padre

```
<div id="divPadre">
    <div name="div" ondblclick="funcion(this)">1</div>
    <div name="div" ondblclick="funcion(this)">2</div>
    <div name="div" ondblclick="funcion(this)">3</div>
    <div name="div" ondblclick="funcion(this)">4</div>
    <div name="div" ondblclick="funcion(this)">5</div>
    <div name="div" ondblclick="funcion(this)">6</div>
    <div name="div" ondblclick="funcion(this)">7</div>
    <div name="div" ondblclick="funcion(this)">8</div>
    <div name="div" ondblclick="funcion(this)">9</div>
    <div name="div" ondblclick="funcion(this)">10</div>
</div>
```

```
let padre = document.getElementById("divPadre")
let hijos = padre.children
```

Devuelve una HTMLCollection, se puede recorrer con un for